PAOO

Effective C++ Items 4–6

Initialization, compiler-generated functions and disallowing copies.

ITEM 4 · INITIALIZATION

Uninitialized Values Are Undefined Behavior

```
int x;   // 0 if global/static, garbage if local

class Point {
    int x, y;
};
Point p; // same rule for p.x, p.y
```

Reading an uninitialized value is **undefined behavior**: garbage values, a crash, or it seems to work.

DECLARED AS	INITIALIZED FOR YOU?
Global or static int	**Yes**, to 0
Local int in a function	**No** guaranteed initial value
int member of a class	**Not always**; depends on how the object is initialized
std::string, std::vector	**Yes**: their own constructor

> The rules are too complicated to memorize, so: **always initialize**. Built-in types by hand (int x = 0;), everything else in the constructor.

ITEM 4 · INITIALIZATION

Assignment Is Not Initialization

ASSIGNMENT IN THE BODY

```
ABEntry::ABEntry(const std::string& name, ...)
{
    theName = name;          // assignments,
    theAddress = address;    // not
    thePhones = phones;      // initializations
    numTimesConsulted = 0;
}
```

MEMBER INITIALIZATION LIST

```
ABEntry::ABEntry(const std::string& name, ...)
  : theName(name),           // these are all
    theAddress(address),      // initializations
    thePhones(phones),
    numTimesConsulted(0)
{}
```

Members come first

On the left, theName is first built empty, then overwritten: two steps.

Built directly

The list builds each member from its argument: one step.

Sometimes required

const members and references must be set in the list (or, since C++11, at the declaration).

ITEM 4 · INITIALIZATION

Initialization Order and Static Objects

The order is fixed

Base classes first, then members **in the order they are declared** in the class, not the order in the list.

```
class A {
    int b;                  // declared first
    int a;
public:
    A(int x) : a(x), b(a) {} // reads a too early
};
```

b reads a before a is initialized: undefined behavior. Reordering the list does not fix it: if b depends on a, declare a first. g++ -Wall warns (-Wreorder).

Static objects in different files

The dynamic initialization order of these global objects in **different .cpp files** is not guaranteed.

```
FileSystem& tfs()
{
    static FileSystem fs;   // created on first call
    return fs;
}
```

Replace the global object with a function that returns a local static. Clients call tfs() instead of tfs.

ITEM 5 · COMPILER-GENERATED FUNCTIONS

What C++ Writes for You

```
class Empty {};
// is essentially the same as:

class Empty {
public:
    Empty() { ... }                             // default constructor
    Empty(const Empty& rhs) { ... }             // copy constructor
    ~Empty() { ... }                            // destructor
    Empty& operator=(const Empty& rhs) { ... } // copy assignment
};

Empty e1;         // default constructor, destructor
Empty e2(e1);     // copy constructor
e2 = e1;          // copy assignment
```

Rules

- Implicitly declared; defined when needed.
- All public and inline.
- No default constructor if you declare **any** constructor.
- Copies are **memberwise**: a pointer is copied as an address (shallow copy).
- The generated destructor does not delete what a pointer member points to.
- C++11 adds the move constructor and move assignment.

When operator= Is Not Generated

```
template<typename T>
class NamedObject {
public:
    NamedObject(std::string& name, const T& value);
private:
    std::string& nameValue;   // reference
    const T objectValue;      // const
};

std::string newDog("Persephone");
std::string oldDog("Satch");
NamedObject<int> p(newDog, 2);
NamedObject<int> s(oldDog, 36);
p = s;  // error: operator= is deleted
```

Why the compiler refuses

- A **reference** cannot be made to refer to a different object.
- A **const** member cannot be changed after initialization.
- So the implicit operator= is defined as deleted, and p = s; does not compile. If you need it, write it yourself.

Copying a new object still works

NamedObject<int> c(p); compiles: the copy constructor initializes, it does not assign.

ITEM 6 · DISALLOWING FUNCTIONS

Disallowing Copies

```
HomeForSale h1;
HomeForSale h2;
HomeForSale h3(h1);   // should not compile
h1 = h2;              // should not compile
```

Every home is unique, so a copy makes no sense. But if you do not declare copying, the compiler generates it (Item 5).

C++98, THE BOOK: PRIVATE

```
class HomeForSale {
public:
    ...
private:
    HomeForSale(const HomeForSale&);
    HomeForSale& operator=(const HomeForSale&);
};
```

Never defined: members and friends get a link error. Or inherit privately from Uncopyable.

C++11 AND LATER: = DELETE

```
class HomeForSale {
public:
    HomeForSale() = default;
    HomeForSale(const HomeForSale&) = delete;
    HomeForSale& operator=(const HomeForSale&) = delete;
};
```

Compile-time error everywhere: “use of deleted function”.

SUMMARY

Things to Remember

ITEM 4

Initialize before use

- Initialize built-in types by hand.
- Use the member initialization list, in declaration order.
- Replace non-local statics with local statics.

ITEM 5

Know what C++ writes

- Default constructor, copy constructor, operator= and destructor.
- Generated copies are memberwise.
- In our example, implicit copy assignment is deleted because of the reference and const int members.

ITEM 6

Forbid what you don’t want

- C++98: declare private, never define, or inherit Uncopyable.
- C++11 and later: = delete.